MOTORMINDER

Project Outline

Ethan Hess, Ethan Kidd, Preston Little, Ryan Carbine

The Backlog Blackhole

Table of Contents

Project Description

This project proposes the development of a car maintenance tracking and recommendation application designed to help vehicle owners better understand, manage, and maintain their vehicles' health. A lot of drivers have a hard time keeping track of routine maintenance, like oil changes, tire rotations, inspections, etc. This can lead to unnecessary repairs, wasted money, and reduced vehicle lifespan. This application aims to consolidate vehicle information to give the user valuable maintenance recommendations based on vehicle condition data.

Software Requirement Specification

Functional Requirements

FR1: Mileage Input

- FR1.1: The system shall allow users to input current vehicle milage as a positive integer.
- FR1.2: The system shall validate that mileage input is numeric and greater than zero.
- FR1.3: The system shall prevent mileage entries less than previously recorded mileage.

FR2: Maintenance Item Management

- FR2.1: The system shall store at least 10 standard maintenance items with their recommended service intervals.
- FR2.2: The system shall allow users to record the mileage at which each maintenance item was last performed.
- FR2.3: The system shall calculate the next due mileage for each maintenance item based on last service mileage + service interval.

FR3: Maintenance Recommendations

- FR3.1: The system shall display all maintenance items that are overdue (current mileage >= next due mileage).
- FR3.2: The system shall display maintenance items due within 1,000 miles of current mileage.
- FR3.3: The system shall categorize recommendations as “Overdue”, “Due Soon”, or “OK”.

FR4: Service History

- FR4.1: The system shall allow users to mark maintenance items as completed with date and mileage.
- FR4.2: The system shall maintain a history of all completed maintenance with timestamps.

FR5: Multiple Vehicles

- FR5.1: The system shall allow users to add up to *4* vehicles to their account.
- FR5.2: The system shall prevent users from adding more than 4 vehicles and display an error message when the limit is reached.
- FR5.3: The system shall require each vehicle to have a unique identifier (e.g., nickname or license plate) within a user’s account.
- FR5.4: The system shall display a list of all vehicles showing vehicle name and current maintenance status for each.

- FR5.5: The system shall allow users to select a specific vehicle to view detailed maintenance recommendations.

FR6: Mechanic Finder

- FR6.1: The system shall retrieve the user's current location coordinates (latitude and longitude) with an accuracy within 1 mile when the mechanic finder feature is accessed.
- FR6.2: The system shall display mechanic finder results only when a vehicle has one or more maintenance items categorized as "Overdue" or "Due Soon"
- FR6.3: The system shall display each mechanic result with the following information: business name, address, distance from the user's current location, and a phone number if available.

Non-Functional Requirements

NFR1: Performance

- NFR1.1: The system shall calculate and display maintenance recommendations within 2 seconds of milage input.
- NFR1.2: The system shall support at least 100 concurrent users without degradation.

NFR2: Usability

- NFR2.1: The system shall be accessible via standard web browsers (Chrome, Firefox, Safari, Edge).
- NFR2.2: Users shall be able to input mileage and view recommendations in 3 clicks or less from the home page.
- NFR2.3: The system shall display error messages that clearly indicate the problem and solution.

NFR3: Reliability

- NFR3.1: The system shall have 99% uptime during business hours (8AM – 8PM).
- NFR3.2: User data shall be persisted and survive the system restarts without loss.

NFR4: Security

- NFR4.1: User accounts require authentication before accessing vehicle data.
- NFR4.2: Passwords shall be encrypted using industry-standard hashing algorithm.

NFR5: Maintainability

- NFR5.1: The system shall use a modular architecture allowing individual component updates.
- NFR5.2: Code shall include inline comments for complex business logic sections.

Design

Design Patterns

MotorMinder is a Python-based CLI application for vehicle maintenance tracking. The project is architected to ensure scalability, maintainability, and clear separation of concerns. This documentation summarizes the design patterns incorporated into the project.

Model-View-Controller (MVC)

- **Model:** Handles data logic and persistence (`models.py`, `data_handler.py`).
- **View:** Manages CLI formatting and user output (`view.py`).
- **Controller:** Bridges user actions and model updates (`controller.py`).
- **CLI:** Entry point for user interaction (`cli.py`).

Benefit: Promotes separation of concerns, easier maintenance, and scalability.

Singleton Pattern

- **Where:** `DataHandler` class in `data_handler.py`.
- **How:** Uses a class-level `_instance` and custom `__new__` method to ensure only one instance exists for data management.

Benefit: Centralizes data access and persistence, prevents conflicting data states.

Command Pattern

- **Where:** `CLI` class in `cli.py`.
- **How:** Maps user menu selections to method calls, acting as a command dispatcher.

Benefit: Simplifies user input handling, makes adding new commands straightforward.

Additional Patterns and Principles

Dataclass/Value Object: `ServiceRecord`, `Vehicle`, and `ServiceStatus` use Python's `@dataclass` for clean, immutable data structures.

Adapter (Implicit): The `Controller` adapts between CLI and model/data layers.

Enum Usage: `ServiceName` enum for type-safe service names.

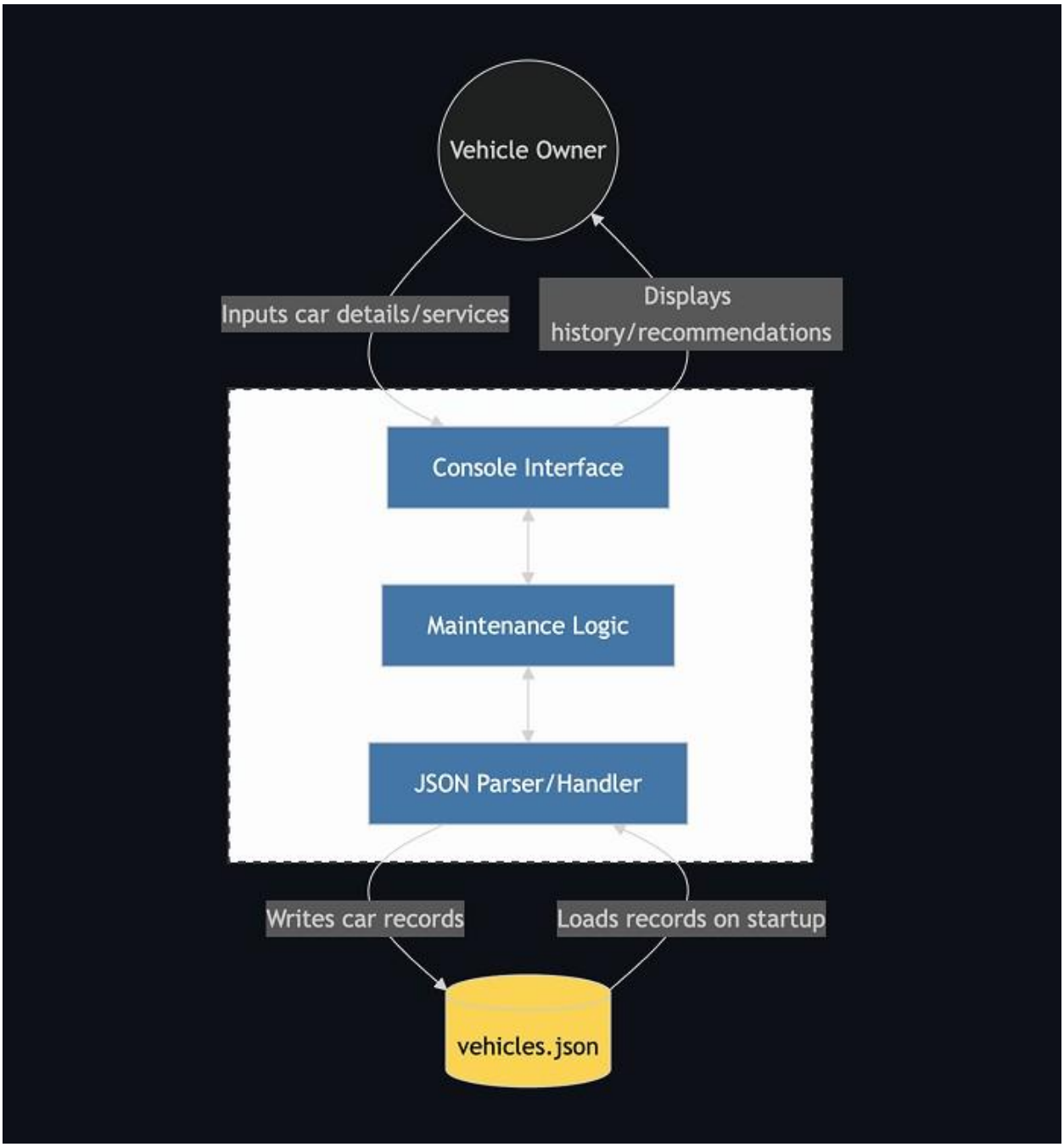
Composition: `Controller` composes a `DataHandler` instance.

Architecture Diagram

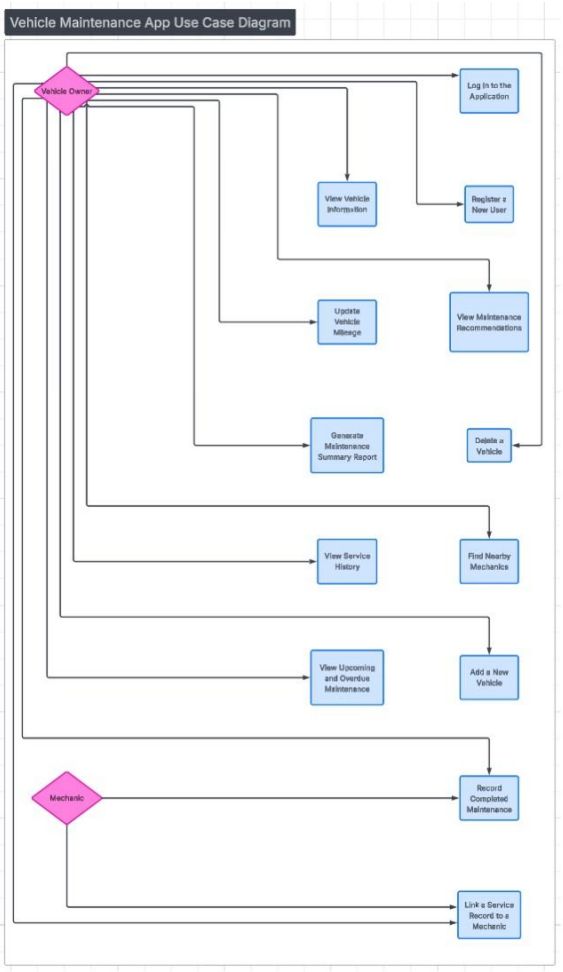
Level 1: System Context Diagram (MVP)

This diagram follows the C4 Model for software architecture, representing the highest level of abstraction. It illustrates how the MotorMinder application interacts with the user and the local environment.

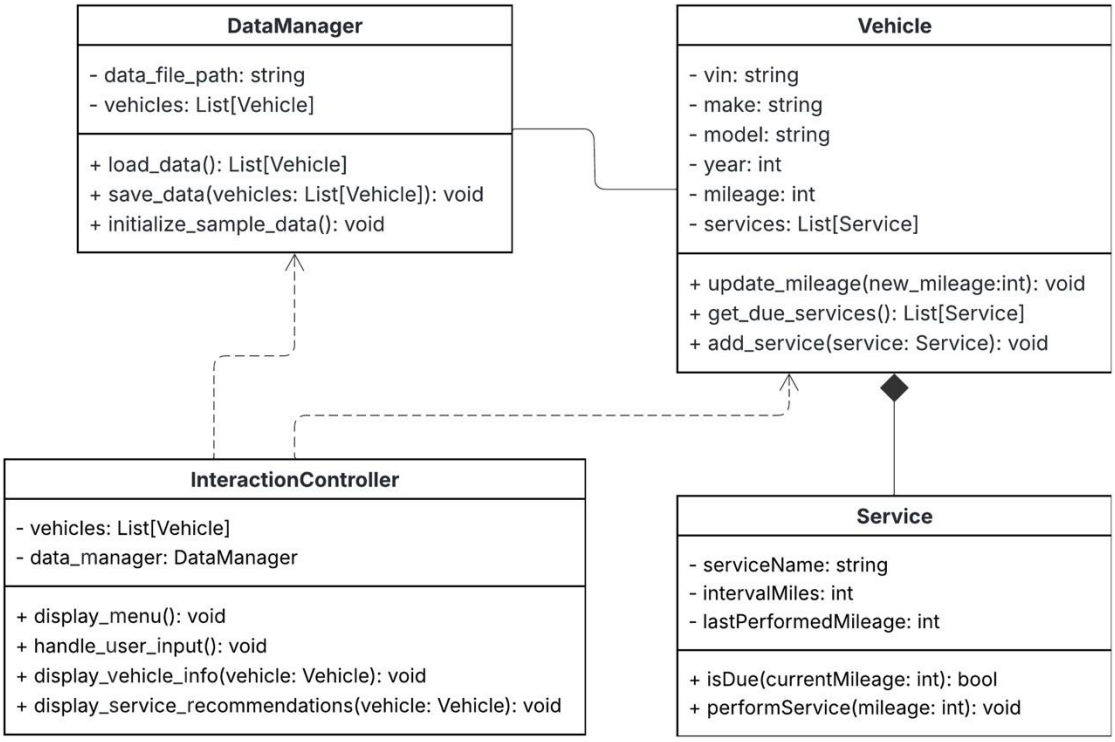
For the Minimum Viable Product (MVP), the system is implemented as a Console-Based Python Application. To ensure simplicity and portability during initial development, the system relies on local JSON persistence rather than external database servers or cloud APIs.


Vehicle Owner
Inputs car details/services
Displays history/recommendations
Console Interface
Maintenance Logic
JSON Parser/Handler
Writes car records
Loads records on startup
vehicles.json

Use Case Diagram



Class Diagram



Testing and QA

Testing Plan

Testing in Sprint 1 focuses on validating the core functionality of the MVP, including data handling, vehicle and service logic, and basic user interaction through the terminal-based prototype. Because the system uses simulated vehicle data, testing is limited to logical correctness rather than real-world accuracy.

Testing Objectives

- Verify that sample vehicle data is loaded and stored correctly
- Ensure maintenance service recommendations are generated based on mileage rules
- Validate correct interaction flow in the terminal interface
- Identify and document defects early in development

Types of Testing

Unit Testing

Unit testing will focus on individual classes and methods, including:

- Vehicle mileage updates
- Service due calculations
- JSON data creation and loading

Unit tests will be manual or lightweight, as automated testing is planned for future sprints.

Integration Testing

Integration testing will verify interactions between major components, including:

- DataHandler loading vehicle data into the system
- Controller accessing vehicle and service information
- Correct data flow between classes

System Testing

System testing will be performed by running the prototype end-to-end to ensure:

- Users can view vehicle information
- Maintenance recommendations are displayed correctly
- The application behaves as expected under normal usage

Test Environment

- **Programming Language:** Python
- **Execution Environment:** Local development machines
- **Data Source:** Simulated JSON vehicle data
- **Interface:** Terminal-based user interface

Defect Tracking

Defects identified during testing will be documented using a standardized bug report template. Each report will include a description of the issue, steps to reproduce, expected behavior, and actual behavior. Bug resolution will be prioritized in future sprints.

Bug Report

Title: Missing Type Annotations, Unused Imports, and Input Validation in CLI and Models

Description: The project contains several code quality issues that, while not causing runtime errors, may impact maintainability and clarity:

- Functions and methods lack type annotations, reducing static analysis and code readability.
- Unused imports are present in `cli.py` and `models.py`, which can clutter the codebase.
- User input conversion (e.g., int(input(...))) lacks error handling, which may cause crashes if invalid input is provided.

- Repeated imports of ServiceName in `cli.py` could be consolidated for efficiency.

Steps to Reproduce:

1. Review the source code in `src/cli.py` and `src/models.py`.
2. Observe missing type annotations, unused imports, and lack of input validation

Expected Behavior:

- All functions and methods should have appropriate type annotations.
- Unused imports should be removed.
- User input should be validated to prevent runtime errors.
- Imports should be consolidated where possible.

Actual Behavior:

- Type annotations are missing.
- Unused imports are present.
- Input validation is not implemented.
- Imports are repeated.

Environment:

- OS: Windows
- Python version: 3.11
- Project location: D:/VSCode/Classes/The-Backlog-Blackhole

Severity: Low (Code quality, not functional bugs)

Priority: Medium (Recommended for maintainability)

Possible Workaround: N/A

Related Issues: N/A

Assignee: Unassigned

Screenshots/Logs: N/A

Quality Assessment Metrics Selection

- **Design Metric:** coupling between modules
- **Testing Metric:** test coverage percentage
- **QA Metric:** defect density
- **Process Metric:** sprint velocity
- **Maintenance Metric:** mean time to fix defects

Customer Feedback

- “There isn’t input validation. For example, if I entered ‘`Year: abc`’ the program crashes. Or if I type ‘`Select vehicle index: 999`’ sometimes it handles it and sometimes it doesn’t.”
- “The mileage logic isn’t explained. In the dashboard, I see ‘OK’, ‘Due Soon’, and ‘Overdue’ but I don’t know what the threshold is, what ‘soon’ means, or what rule triggered it.”
- “When I edit mileage or make it says updated, but I don’t see the new full vehicle details. I’d expect it to reprint the updated vehicle.”
- “Logging a service doesn’t validate the date. If I enter ‘`Service data: banana`’ it’ll likely break when parsed later.”